

Design Note — TaskTracker CLI App

Overview

TaskTracker es una aplicación de línea de comandos diseñada para gestionar tareas personales de forma simple y eficiente. Permite crear, listar, marcar como completadas y eliminar tareas. Aunque el proyecto es pequeño, está pensado con criterios de diseño que conectan algoritmos, estructuras de datos, manejo de memoria y decisiones de arquitectura para lograr un sistema claro, mantenible y con buen rendimiento.

Objetivos del diseño

1. Mantener el código modular y fácil de extender.
 2. Elegir estructuras de datos que ofrezcan operaciones rápidas.
 3. Minimizar el uso innecesario de memoria.
 4. Mantener la complejidad temporal lo más baja posible en las operaciones críticas.
 5. Asegurar que la persistencia de datos sea simple pero efectiva.
-

Estructura general del sistema

El sistema está dividido en tres módulos principales:

1. **task_manager.py**
 - Contiene la lógica principal de manejo de tareas.
 - Se encarga de agregar, eliminar, actualizar y listar.
2. **storage.py**
 - Encapsula la lectura y escritura de disco.
 - Permite cambiar el backend de almacenamiento sin modificar la lógica central.

3. cli.py

- Gestiona la interacción con el usuario.
- Traduce los comandos del usuario en llamadas al `TaskManager`.

Este enfoque modular mejora la mantenibilidad y reduce el acoplamiento, lo que facilita las pruebas unitarias y futuras expansiones (por ejemplo, pasar de CLI a web sin reescribir la lógica interna).

Elección de estructuras de datos

La elección principal es usar una **lista de diccionarios** para almacenar las tareas en memoria:

```
[  
  { "id": 1, "title": "Comprar comida", "done": False },  
  { "id": 2, "title": "Estudiar Java", "done": True }  
]
```

Motivación

- Mantener una lista permite:
 - Iterar en orden de creación (útil al mostrar tareas).
 - Insertar al final en $O(1)$.
 - Serialización simple a JSON.
- Usar diccionarios para representar tareas:
 - Mejora la legibilidad.
 - Facilita extensiones futuras (fechas, etiquetas, prioridad).
 - Minimiza el overhead comparado con clases más complejas.

Costos y complejidad

- **Agregar tarea:** $O(1)$ amortizado.

- **Eliminar por ID:** $O(n)$, ya que se requiere búsqueda lineal.
→ Para este tamaño de app, n es pequeño y el coste aceptable.
- **Marcar como completa:** $O(n)$ por la misma razón.
- **Listar:** $O(n)$.

¿Por qué no un diccionario global `{id: tarea}`?

Porque:

- La CLI normalmente muestra las tareas en orden.
- Priorizar legibilidad y simplicidad es mejor para el contexto.
- No se necesita acceso aleatorio constante.

Si el proyecto creciera, se podría migrar a:

- Un diccionario hash para búsquedas $O(1)$.
- Una base de datos ligera como SQLite.

Decisiones de memoria

La aplicación mantiene todas las tareas en memoria **solo mientras está en ejecución**, y luego las vuelca a un archivo JSON.

Razones:

- Evita uso innecesario de memoria persistente en RAM.
- JSON es legible para debug y edición manual.
- Es portable entre sistemas.

El uso de diccionarios y listas implica un overhead moderado, pero aceptable para una app pequeña. Cada tarea ocupa solo unos pocos bytes adicionales y permite manipulación rápida sin parsing complejo.

Persistencia

Los datos se guardan en `tasks.json`.

Ventajas

- Serialización/deserialización eficiente.
- Evita dependencias externas.
- Permite que el usuario haga backups manuales.

Desventajas

- Operaciones de escritura son $O(n)$ al reescribir el archivo completo.
→ Válido para una app pequeña, pero en un futuro se podría optimizar.
-

Interfaz de usuario

La CLI ofrece comandos como:

```
task add "Estudiar Java"
task list
task done 2
task delete 1
```

El diseño favorece:

- Comandos intuitivos.
 - Respuestas claras.
 - Errores manejados con mensajes explicativos.
-

Conclusión

Aunque TaskTracker es un proyecto pequeño, fue diseñado aplicando pensamiento de sistema: se conectan estructuras de datos, costos computacionales, decisiones de memoria

y diseño modular. Esto permite que la aplicación sea clara, robusta y fácil de extender, manteniendo un rendimiento adecuado para su escala.

Si en el futuro creciera, la arquitectura facilita:

- Migrar a SQLite.
- Añadir una API REST.
- Implementar indexación más eficiente.
- Añadir etiquetas, fechas o usuarios.